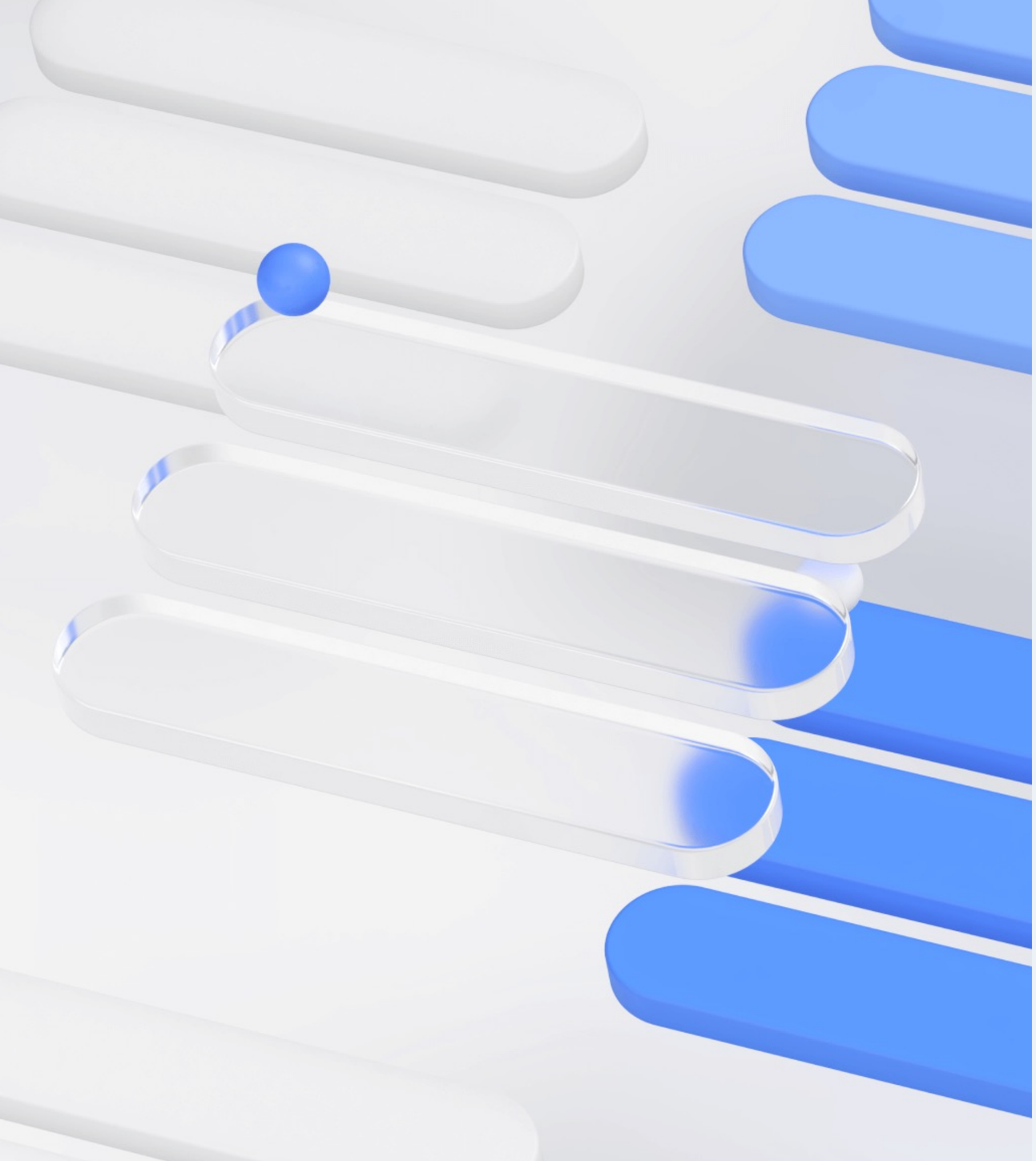
Yandex Cloud

Облачные технологии в работе с большими данными. День 13

Баташов Руслан

Serverless

1. Введение
2. Cloud Functions
3. API Gateway
4. Serverless YDB
5. Yandex Message Queue



Serverless

Бессерверные вычисления, которые помогают разрабатывать приложения, хранить данные и настраивать интеграцию с другими платформами без создания виртуальных машин и обслуживания инфраструктуры.

Serverless

Использование serverless-сервисов при разработке приложений даёт следующие преимущества:

1. Позволяет разработчикам сфокусироваться на работе над кодом. Больше не нужно разворачивать, настраивать и обслуживать какую-либо инфраструктуру (серверы, виртуальные машины, контейнеры).

2. Часто даёт возможность снизить операционные расходы по сравнению с запуском приложений на виртуальной машине или в средах контейнерной виртуализации — за счёт более эффективного распределения мощностей провайдером.

3. Позволяет минимизировать риски непредсказуемости спроса — в случае как переоценки, так и недооценки будущей нагрузки. А значит, становится проще принимать решения о добавлении в приложение новых функций или о запуске рекламной кампании.

Serverless

1. Введение
2. Cloud Functions
3. API Gateway
4. Serverless YDB
5. Yandex Message Queue

Yandex Cloud Functions

Относится к категории Function-As-A-Service (FaaS), его главная задача — исполнять программный код, который написан на одном из поддерживаемых языков программирования (сейчас их восемь).
Для запуска кода облачных (бессерверных) функций используются специально созданные виртуальные машины, на которых установлены рантаймы.

Однако сервис Cloud Functions умеет не только это. Он также:

1. **Ведёт журнал выполнения функций.** Сюда выводятся параметры и время запуска функции, а также время и результат её выполнения. У пользователя есть возможность выводить в журнал и собственную информацию. Это удобно как при создании и отладке функций, так и при разборе нештатных ситуаций. Например, данные, которые вам нужно было записать в базу данных, в итоге туда не попали. Почему? Не вызвалась функция или была недоступна база данных? В коде функции была ошибка? Журнал поможет найти ответы на эти вопросы.

2. **Интегрируется с другими сервисами Yandex Cloud.** Для этого он использует триггер — механизм, который позволяет запускать функцию по событиям в других сервисах. Например, вы можете вызывать функцию при получении пакета данных от ваших IoT-устройств для его обработки.

3. **Предоставляет SDK для разработчиков.** Это программные модули, которые можно подключить и вызвать из пользовательских функций. Они упрощают взаимодействие с другими сервисами Yandex Cloud из кода программы. Например, вы можете использовать SDK для сохранения данных в Yandex Object Storage.

Yandex Cloud Functions. Ограничения

1. **Функции должны быть конечными.** Они не могут бесконечно выполнять какую-то задачу. Это ограничение заложено в суть самого сервиса, есть даже специальный таймер, по истечении которого функция будет автоматически завершена.

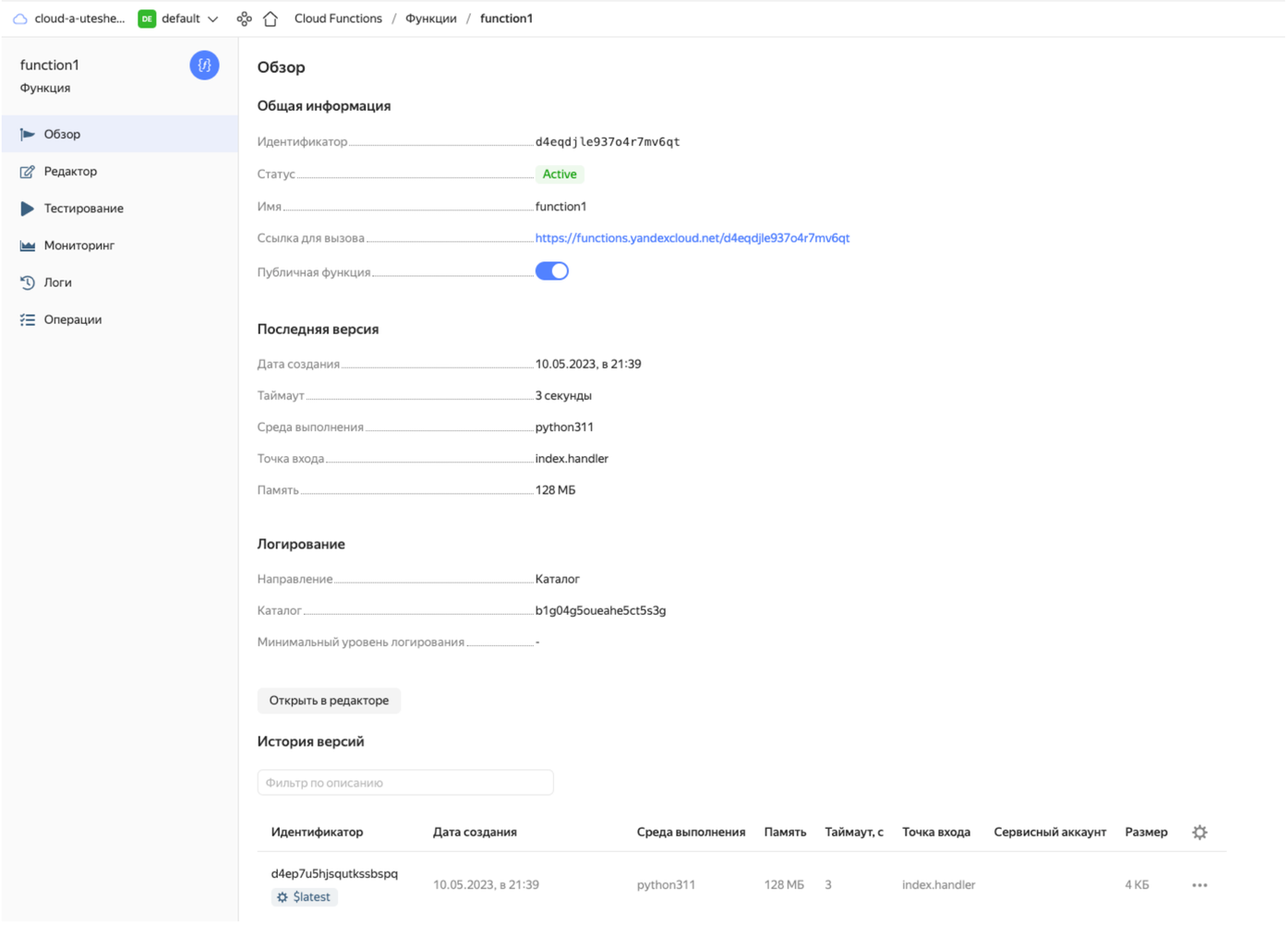
2. **Функции не могут сами по себе хранить состояния.** Cloud Functions — среда выполнения, в которой не сохраняется состояние. Каждая запущенная копия (instance) функции отвечает только за один запрос одновременно (в случае большого количества запросов эта копия может последовательно их обработать). Конкурирующие запросы обрабатываются разными копиями функции, эти копии не могут обмениваться переменными или локальной памятью. Если вам нужно, чтобы функция сохраняла состояние, данные нужно записать в файл в объектном хранилище или в базу данных, например YDB.

Создании функции

1. На главной странице консоли управления в списке сервисов выберите **Cloud Functions**
2. На открывшейся странице нажмите кнопку **Создать функцию**:
3. Укажите имя функции, введите короткое описание того, что она будет делать, и нажмите кнопку **Создать**:
4. Затем выберите среду выполнения кода и нажмите кнопку **Продолжить**:
5. По умолчанию сервис предлагает создать Hello World — файл с примером кода на выбранном языке программирования. Этот файл будет создан и автоматически загружен в контейнер. В поле **Способ** укажите **Редактор кода** и выберите файл index.go.

По умолчанию сервис предлагает работать с редактором кода прямо в веб-интерфейсе. Однако вместо этого вы можете загрузить файл с кодом из бакета Object Storage (этот способ подойдёт для файлов больше 3,5 МБ) или загрузить ZIP-архив с кодом с локальной машины. Переключатель способа добавления кода находится прямо над окном редактора.
Код вашей функции может находиться как в одном файле, так и в нескольких. Вы также можете создавать папки. При этом обязательно нужно указывать **точку входа** — часть кода, которая будет вызываться первой и принимать параметры вызова. Формат точки входа — <имя файла с функцией>.<имя обработчика вызова>. Например, index.Handler.

6. Вверху справа нажмите кнопку **Создать версию**, чтобы сохранить текущее состояние функции. Сервис создаст версию функции и покажет справочную страницу о ней.



7. Теперь в панели слева перейдите на вкладку **Тестирование**. В поле **Шаблон данных** выберите HTTPS-вызов. Сервис автоматически сгенерирует входные данные в формате JSON.
8. Под полем с входными данными нажмите кнопку **Запустить тест**. Сервис выполнит HTTPS-вызов созданной функции и сформирует ответ (также в формате JSON). - Вы запустили свою первую облачную функцию.

Результат тестирования

Состояние функции **Выполнена**

Ответ функции

```
{
    "statusCode": 200,
    "body": "Hello World!"
}
```

Логи

Время (UTC +3)	Уровень	JSON	Сообщение
21:42:53, 10.05.2023	INFO		REPORT RequestID: e8e0b032-3f0d-4093-af42-9e91904c0d40 Duration: 6.607 ms Billed Duration: 100 ms Memory Size: 128 MB Max Memory Used: 54 MB Queuing Duration: 0.023 ms Function Init Duration: 4.713 ms
21:42:53, 10.05.2023	INFO		END RequestID: e8e0b032-3f0d-4093-af42-9e91904c0d40
21:42:53, 10.05.2023	INFO		START RequestID: e8e0b032-3f0d-4093-af42-9e91904c0d40 Version: d4ep7u5hjsqutkssbspq

Запуск функции через CLI

1. Создайте сервисный аккаунт с именем **service-account-for-cf**
2. Добавьте вновь созданному сервисному аккаунту роль editor
3. Создайте функцию с именем **my-first-function**:

yc serverless function create --name my-first-function

Вы получите URL, по которому можно будет сделать вызов функции http_invoke_url. По умолчанию функция будет непубличной.

```
ipivnev@ipivnev-osx ~ % yc serverless function create --name my-first-function
done (1s)
id: d4eol020jhfcfb1moubk
folder_id: b1g04g5oueahe5ct5s3g
created_at: "2023-05-11T21:02:10.486Z"
name: my-first-function
log_group_id: ckgubfbn8u0d8ofnev59
http_invoke_url: https://functions.yandexcloud.net/d4eol020jhfcfb1moubk
status: ACTIVE

There is a new yc version '0.104.0' available. Current version: '0.102.0'.
See release notes at https://cloud.yandex.ru/docs/cli/release-notes
You can install it by running the following command in your shell:
        $ yc components update
```

Загрузка кода функции

1. Создайте файл index.py :

```
sudo nano index.py
```

2. Добавьте в index.py следующее содержимое:

```
def handler(event, context):
return {
'statusCode': 200,
'body': 'Hello World!',
}
```

Успешное выполнение этой функции вернёт небольшую веб-страницу.

3. Загрузите код функции в облако и создайте её версию. Для этого перейдите в папку с файлом index.py и выполните команду:

```
yc serverless function version create \
  --function-name my-first-function \
  --memory 256m \
  --execution-timeout 5s \
  --runtime python37 \
  --entrypoint index.handler \
  --service-account-id $SERVICE_ACCOUNT_ID \
  --source-path index.py
```

Успешное выполнение команды приведёт к созданию версии функции. С помощью консоли управления убедитесь, что версия создана.

4. Получите список функций, а затем — информацию о функции my-first-function:

```
yc serverless function list
```

```
yc serverless function version list --function-name my-first-function
```

5. В результате вызова последней команды из столбца FUNCTION ID вы узнаете идентификатор функции и сможете сделать вызов функции с помощью следующей команды:

```
yc serverless function invoke <идентификатор_функции>
```

```
ipivnev@ipivnev-osx Documents % yc serverless function list
+----------------------+--------------------+----------------------+--------+
|          ID          |       NAME         |      FOLDER ID       | STATUS |
+----------------------+--------------------+----------------------+--------+
| d4eol020jhfcfb1moubk | my-first-function | b1g04g5oueahe5ct5s3g | ACTIVE |
| d4eqdjle937o4r7mv6qt | function1          | b1g04g5oueahe5ct5s3g | ACTIVE |
+----------------------+--------------------+----------------------+--------+

ipivnev@ipivnev-osx Documents % yc serverless function version list --function-name my-first-function
+----------------------+----------------------+----------+---------------+---------+----------------------+
|          ID          |     FUNCTION ID      | RUNTIME  |  ENTRYPOINT   |  TAGS   |     CREATED AT       |
+----------------------+----------------------+----------+---------------+---------+----------------------+
| d4e4024k6gi10698d3u8 | d4eol020jhfcfb1moubk | python37 | index.handler | $latest | 2023-05-12 17:20:18 |
+----------------------+----------------------+----------+---------------+---------+----------------------+
```

6. По умолчанию функция создаётся непубличной. Чтобы сделать функцию my-first-function публичной, выполните следующую команду:

```
yc serverless function allow-unauthenticated-invoke my-first-function
```

7. После этого вы сможете вызвать её в браузере. Получите параметр http_invoke_url для функции my-first-function:

```
yc serverless function get my-first-function
```

8. Введите значение параметра http_invoke_url в браузере и наслаждайтесь вызовом вашей функции.

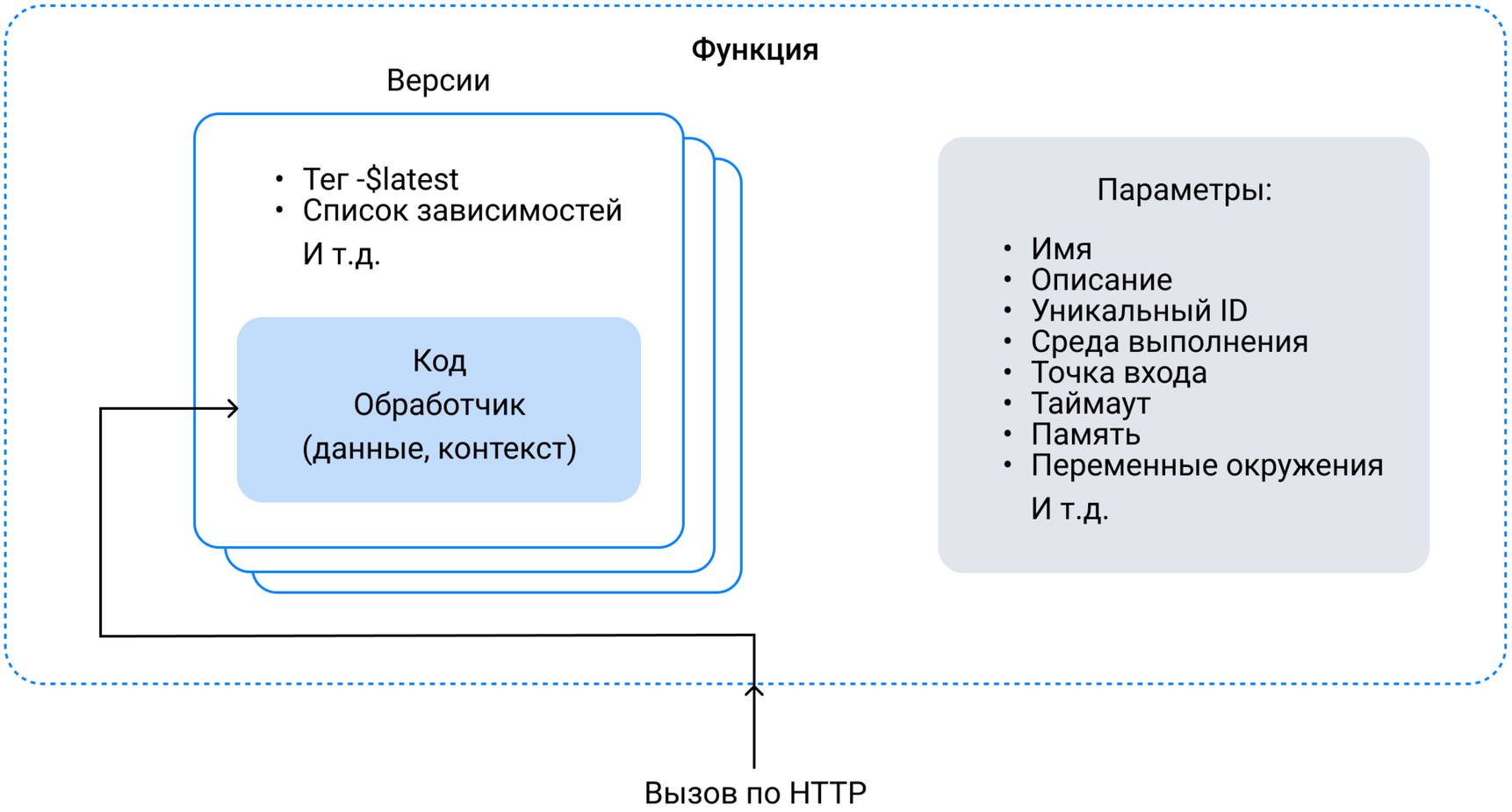
Как готовить функции к запуску и управлять ими

Логика работы с функциями напоминает работу с контейнерами. Только что созданная функция — набор данных, который позволяет её распознать по уникальному идентификатору, имени и описанию. Этот набор также содержит тип среды выполнения, переменные окружения и параметры вызова кода. При этом кода, который будет запускаться в только что созданной функции, нет.

Функция представляет собой логический контейнер с набором параметров. Внутри контейнера вызывают **версию** пользовательского кода, которая обязательно содержит **метод**. Его нужно указать в точке входа функции. **Обработчик** принимает поступающие на вход данные и вызывает необходимые функции и методы в коде. Он должен соответствовать требованиям модели программирования функций и уметь принимать данные и контекст вызова.

Контекст используют для взаимодействия программного кода функции с сервисом Yandex Cloud Functions. Например, с помощью контекста узнают время до завершения выполнения Cloud Functions.
Ещё одно важное понятие модели программирования — **среда выполнения**, т. е. логическая инфраструктура, которая содержит необходимое окружение для запуска пользовательского кода. Она соответствует выбранному языку программирования.

Как готовить функции к запуску и управлять ими



В Cloud Functions реализовано простое управление версиями исходного кода. Любое текущее состояние кода вы можете сохранить как новую версию. Для этого после внесения изменений справа вверху на странице редактора нажмите кнопку **Создать версию**.

Для каждой версии сохраняется свой уникальный идентификатор.

Если в новой версии кода оказались ошибки, вы всегда можете оперативно перенастроить **триггер** на выполнение более старой, проверенной версии исходников функции. Для этого используется механизм тегов, позволяющий вызывать определённую версию функции.

Последняя версия функции при создании автоматически получает специальный тег $latest, а с предыдущей версии функции этот тег автоматически снимается.

Как готовить функции к запуску и управлять ими

1. Создайте еще одну версию функции по команде с 12го слайда.
2. Проверьте в консоли, что у вашей функции появилась новая версия
3. Добавьте новый тег самой первой версии кода

```
yc serverless function version set-tag \
  --id d4e4024k6gi10698d3u8\
  --tag first
```

Проверьте консоль, что появился новый тег.

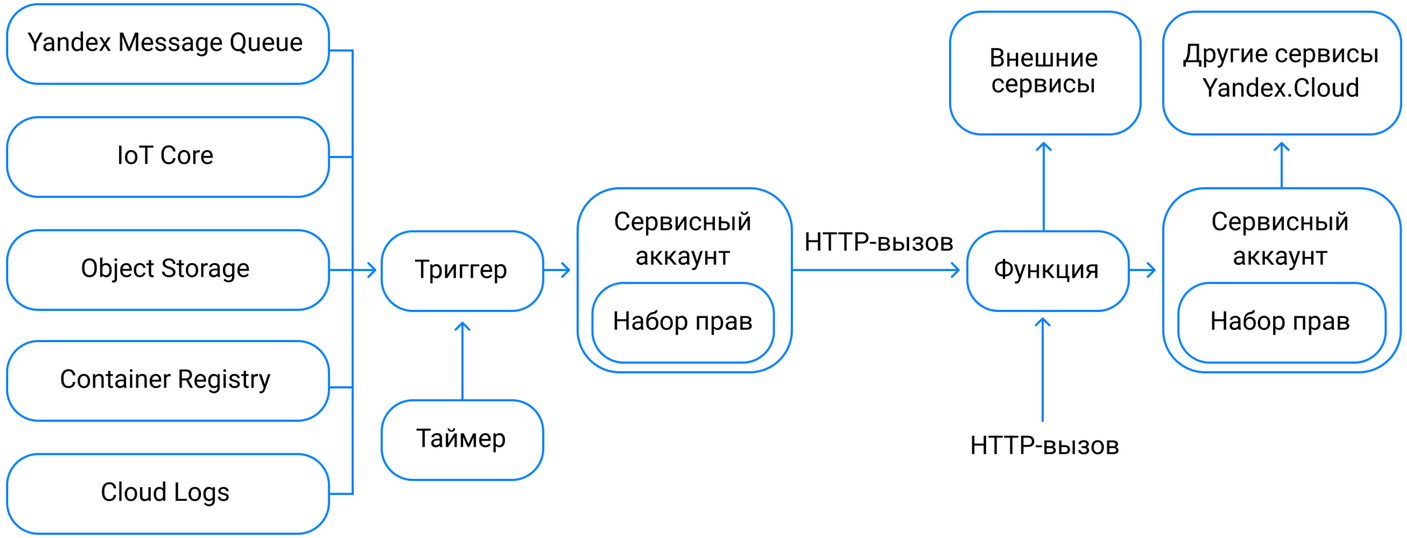
Идентификатор	Дата создания	Среда выполнения	Память	Таймаут, с	Точка входа	Сервисный аккаунт	Размер
d4e73as7flqiqelbuo30 $latest	12.05.2023, в 20:46	python37	256 МБ	5	index.handler	service-account-for-cf	4 КБ
d4e4024k6gi10698d3u8 first	12.05.2023, в 20:20	python37	256 МБ	5	index.handler	service-account-for-cf	4 КБ

Инструменты работы с функциями разнесены по разным вкладкам боковой панели слева. Давайте коротко пройдёмся по ним.

- **Обзор.** Здесь находится сводка самой важной информации о функции: идентификатор, статус, имя, описание, ссылка для вызова, список версий и т.д.
- **Редактор.** Вы можете поменять среду выполнения, внести правки в исходный код, указать переменные окружения, изменить объём памяти, выделяемой для выполнения функции, и т.д.
- **Тестирование.** Здесь вы выбираете версию кода по тегу, готовый шаблон данных или свой собственный — со структурой запроса в формате JSON — и смотрите ответ, который вам возвращает функция.
- **Мониторинг.** Вы можете отслеживать такие показатели, как количество запросов, время задержки откликов на запросы и количество ошибок.
- **Логи.** Здесь вы можете посмотреть подробный журнал о выполнении функции. Он помогает находить ошибки в коде.
- **Операции.** На этой странице перечислены все операции с функцией, такие как обновление кода, назначение и удаление тегов и т.д.

Триггеры

В терминологии Yandex Cloud триггером является некое условие, при выполнении которого автоматически запускается определённая функция. Общая схема работы выглядит следующим образом:



Триггеры

Как видите, триггеры позволяют автоматизировать работу с другими сервисами Yandex Cloud и вызывают функцию:

- **Message Queue** — при появлении нового сообщения для обработки.
- **IoT Core** — при получении посылки с данными от устройства.
- **Object Storage** — при создании, редактировании ACL или удалении объекта.
- **Container Registry** — при создании или удалении Docker-образа или тега Docker-образа.

Создание триггера от Object Storage

1. Для работы с Object Storage добавьте этому сервисному аккаунту роль storage.editor
2. Создайте ключ доступа для вашего сервисного аккаунта

```
yc iam access-key create --service-account-name service-account-for-cf
```

В результате вы получите:

```
ipivnev@ipivnev-osx Documents % yc iam access-key create --service-account-name service-account-for-cf
access_key:
  id: ajeoios353kbtji2aa3b
  service_account_id: ajeb87foth6tmk74voue
  created_at: "2023-05-12T18:04:42.867070375Z"
  key_id: YCAJEiobB2ot84Tl2Uu5exR8e
secret: YCO-VuYXek-e6L79KpwK0CH8jPhlg0yNLKpY8xI5
```

Где:
- key_id — идентификатор ключа доступа, ACCESS_KEY.
- secret — секретный ключ, SECRET_KEY.

Создание триггера от Object Storage

1. В консоли управления в вашем рабочем каталоге выберите сервис Object Storage. Нажмите кнопку **Создать бакет**.

На странице создания бакета:

- Введите имя бакета
- При необходимости ограничьте максимальный размер бакета, установив значение, например, 1 ГБ.
- Выберите тип доступа, в нашем уроке установим значения в **Публичный** во всех случаях.
- Выберите класс хранилища, по умолчанию используется **Стандартное**.
- Нажмите кнопку **Создать бакет** для завершения операции. Далее вы всегда сможете поменять класс хранилища, его размер и настройки доступа.

Создание триггера от Object Storage

Загрузка кода новой версии

2. Новая версия функции имеет зависимости, которые описаны в файле requirements.txt, а это значит, что для загрузки функции в облако необходимо файлы index.py и requirements.txt заархивировать и получить файл my-first-function.zip. Новая версия index.py:

```
import os
import datetime
import boto3
import pytz

ACCESS_KEY = os.getenv("ACCESS_KEY")
SECRET_KEY = os.getenv("SECRET_KEY")
BUCKET_NAME = os.getenv("BUCKET_NAME")
TIME_ZONE = os.getenv("TIME_ZONE", "Europe/Moscow")

TEMP_FILENAME = "/tmp/temp_file"
TEXT_FOR_TEMP_FILE = "This is text file"

def write_temp_file():
    temp_file = open(TEMP_FILENAME, 'w')
    temp_file.write(TEXT_FOR_TEMP_FILE)
    temp_file.close()
    print("\U0001f680 Temp file is written")

def get_now_datetime_str():
    now = datetime.datetime.now(pytz.timezone(TIME_ZONE))
    return now.strftime('%Y-%m-%d__%H-%M-%S')

def get_s3_instance():
    session = boto3.session.Session()
    return session.client(
        aws_access_key_id=ACCESS_KEY,
        aws_secret_access_key=SECRET_KEY,
        service_name='s3',
        endpoint_url='https://storage.yandexcloud.net'
    )

def upload_dump_to_s3():
    print("\U0001F4C2 Starting upload to Object Storage")
    get_s3_instance().upload_file(
        Filename=TEMP_FILENAME,
        Bucket=BUCKET_NAME,
        Key=f'file-{get_now_datetime_str()}.txt'
    )
    print("\U0001f680 Uploaded")


def remove_temp_files():
    os.remove(TEMP_FILENAME)
    print("\U0001F44D That's all!")

def handler(event, context):
    write_temp_file()
    upload_dump_to_s3()
    remove_temp_files()
    return {
        'statusCode': 200,
        'body': 'File is uploaded',
    }
```

Создание триггера от Object Storage

3. Первая версия requirements.txt:

```
boto3==1.13.10
botocore==1.16.10
python-dateutil==2.8.1
pytz==2020.1
```

4. Создайте функцию:

```
yc serverless function version create \
--function-name my-first-function \
--memory 256m \
--execution-timeout 5s \
--runtime python37 \
--entrypoint index.handler \
--service-account-id $SERVICE_ACCOUNT_ID \
--source-path my-first-function.zip \
--environment ACCESS_KEY=$ACCESS_KEY \
--environment SECRET_KEY=$SECRET_KEY \
--environment BUCKET_NAME=$BUCKET_NAME
```

Создание триггера от Object Storage

5. Проверьте создание функции

```
yc serverless function version list --function-name my-first-function
```

```
yc serverless function invoke <FUNCTION ID>
```

6. Вызовите функцию с помощью команды

```
yc serverless function invoke <идентификатор_функции>
```

7. Получите параметр http_invoke_url для функции my-first-function

```
yc serverless function get my-first-function
```

8. Введите значение параметра http_invoke_url в браузере и наслаждайтесь вызовом вашей функции.
   Во время её вызова в Object Storage будет создан новый файл.

Создание триггера

1. Для создания триггера нам необходима функция, которую триггер будет запускать. Аналогично предыдущему шагу создадим функцию my-trigger-function и её версию на основе файла index.py.

```
def handler(event, context):
    print("\U0001F4C2 Starting function after trigger")
    print(event)
    return {
        'statusCode': 200,
        'body': 'File is uploaded',
    }
```

Создание триггера от Object Storage

2. Находясь в каталоге с файлом index.py, вызовите следующие команды:

```
yc serverless function create --name my-trigger-function

yc serverless function version create \
--function-name my-trigger-function \
--memory 256m \
--execution-timeout 5s \
--runtime python37 \
--entrypoint index.handler \
--service-account-id $SERVICE_ACCOUNT_ID \
--source-path index.py

yc serverless function version list --function-name my-trigger-function
```

Создание триггера от Object Storage

3. Чтобы создать триггер my-first-trigger, который вызывает функцию my-trigger-function при создании нового объекта в бакете BUCKET_NAME, выполните команду:

```
yc serverless trigger create object-storage \
--name my-first-trigger \
--bucket-id $BUCKET_NAME \
--events 'create-object’ \
--invoke-function-name my-trigger-function \
--invoke-function-service-account-id $SERVICE_ACCOUNT_ID
```

Вызов цепочки событий

1. Чтобы запустить цепочку событий, вызовем первую функцию my-first-function. Получите список функций и информацию о функции my-first-function:

```
yc serverless function list
```

```
yc serverless function version list --function-name my-first-function
```

2. В результате вызова последней команды в столбце FUNCTION ID вы узнаете идентификатор функции и сможете сделать вызов функции с помощью команды:

```
yc serverless function invoke <идентификатор_функции>
```

3. После этого вы можете сделать её вызов в браузере. Получите параметр http_invoke_url для функции my-first-function

```
yc serverless function get my-first-function
```

Создание триггера от Object Storage

4. Введите значение параметра http_invoke_url в браузере.
   Во время вызова функции в Object Storage будет создан новый объект. Сразу после этого сработает триггер my-first-trigger, который вызовет функцию my-trigger-function. В итоге, наша вторая функция запишет в логи содержание переменной event. Убедиться в этом вы сможете как в UI, так и через CLI.

yc serverless function logs my-trigger-function

```
2023-05-12 20:17:12  INFO START RequestID: 5368bcd7-1c27-4187-82af-c8e9a25abd4c Version: d4e2824gmto3s3psri4m
2023-05-12 20:17:12       📂 Starting function after trigger
2023-05-12 20:17:12       {'messages': [{'event_metadata': {'event_id': 'b567d1e7-18cf-4048-91fe-8a1efaf97c1c', 'event_type': 'yandex.cloud.events.storage.ObjectCreate', 'created_at
': '2023-05-12T20:17:11.114414889Z', 'tracing_context': {'trace_id': '64fa3c1a66961fbd', 'span_id': '', 'parent_span_id': ''}, 'cloud_id': 'b1gbn6daubivtdssfi47', 'folder_id': 'b1g0
4g5oueahe5ct5s3g'}, 'details': {'bucket_id': 'bucket-for-cf', 'object_id': 'file-2023-05-12__23-17-11.txt'}}]}
2023-05-12 20:17:12  INFO END RequestID: 5368bcd7-1c27-4187-82af-c8e9a25abd4c
2023-05-12 20:17:12  INFO REPORT RequestID: 5368bcd7-1c27-4187-82af-c8e9a25abd4c Duration: 176.211 ms Billed Duration: 200 ms Memory Size: 256 MB Max Memory Used: 11 MB Queuing Dura
tion: 0.058 ms Function Init Duration: 173.771 ms
```